

Intent Understanding

1. Course Content

- Master the ability to customize unique user intent understanding functions through the RAG knowledge base.

[!IMPORTANT]

- Intent understanding is designed to increase the rapport between the robot and the user, allowing the robot to better understand the user. This function should not be used to perform "strange" and "unconventional" tasks.

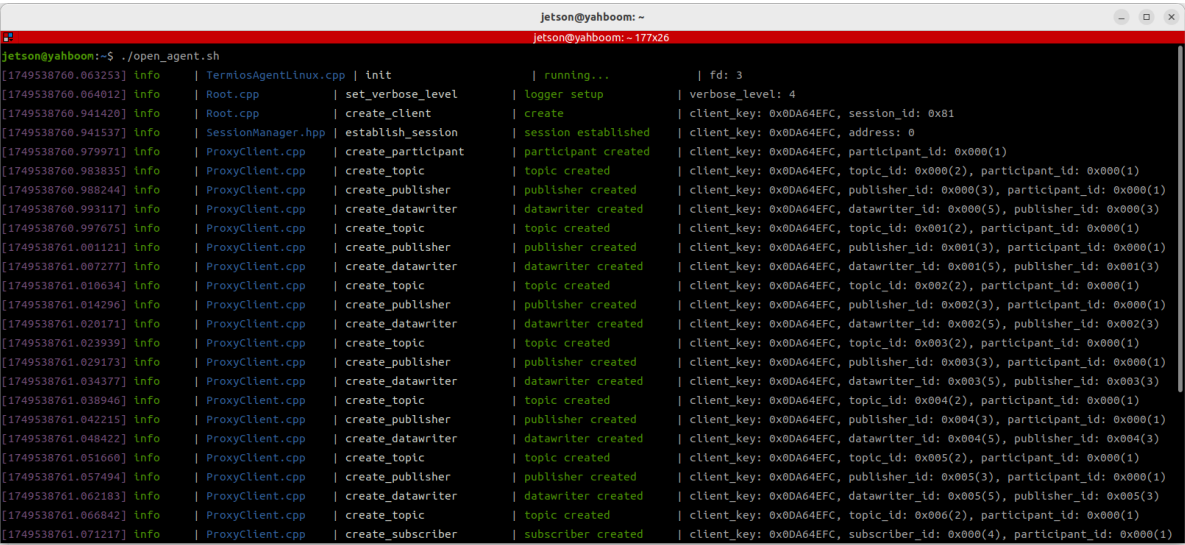
2. Starting the Agent

Note: If already started, no need to start again.

Enter the following command in the vehicle terminal:

```
sh start_agent.sh
```

The terminal will print the following information, indicating a successful connection:



2.2 Configuring the Intent Mapping File

- This file is used to store personal fuzzy intents and the corresponding tasks the robot should perform.
- Open the example file in this tutorial folder. You can add multiple custom intents according to the reference format. The following is a simple example:

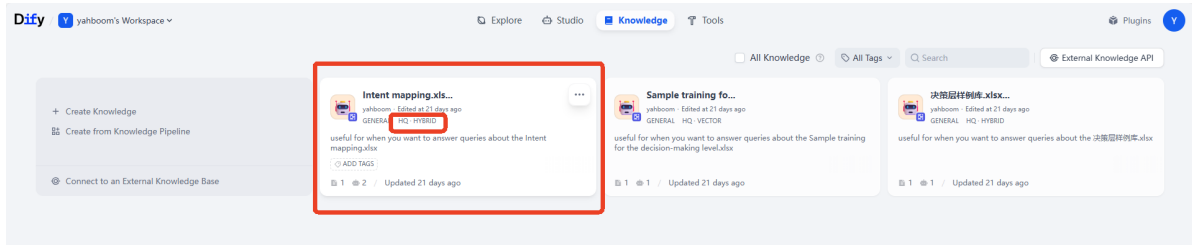
query	answer
I'm a little thirsty	1. Navigate to the kitchen, 2. Check for bottled water or drinks, 3. If available, use the robotic arm to pick up the drink, 4. Navigate back to the starting position

2.3 Configuring the Knowledge Base

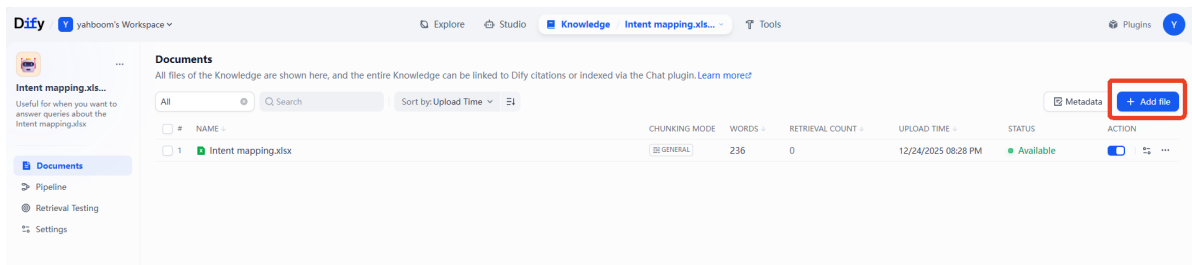
- Next, we need to upload the edited intent mapping file to the **Di fy** **RAG** knowledge base.

[!NOTE]

- For detailed instructions on the RAG knowledge base, please refer to the tutorial in [2. AI Model Development - 06 - Deploy the RAG knowledge base]. - Dify comes pre-configured with a sample knowledge base for **Intent mapping** to demonstrate how to use the intent understanding feature.



- You can modify the **Intent mapping.xls** template, or you can delete it and add your own file.



[!TIP]

It's worth noting:

To ensure optimal intent understanding performance, it is recommended to set the **Intent mapping** knowledge base to high-quality mode, as intent understanding often requires retrieving relevant snippets from semantically similar cues. ## 4. Running Examples

4.1 Starting the Program

- On the vehicle's onboard computer, open a terminal and enter the command to start the AI agent function:

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode
```

- Alternatively, you can use the shortcut command:

```
multi_brains
```

- On the vehicle's onboard computer, open two more terminals and enter the commands to start the navigation function:

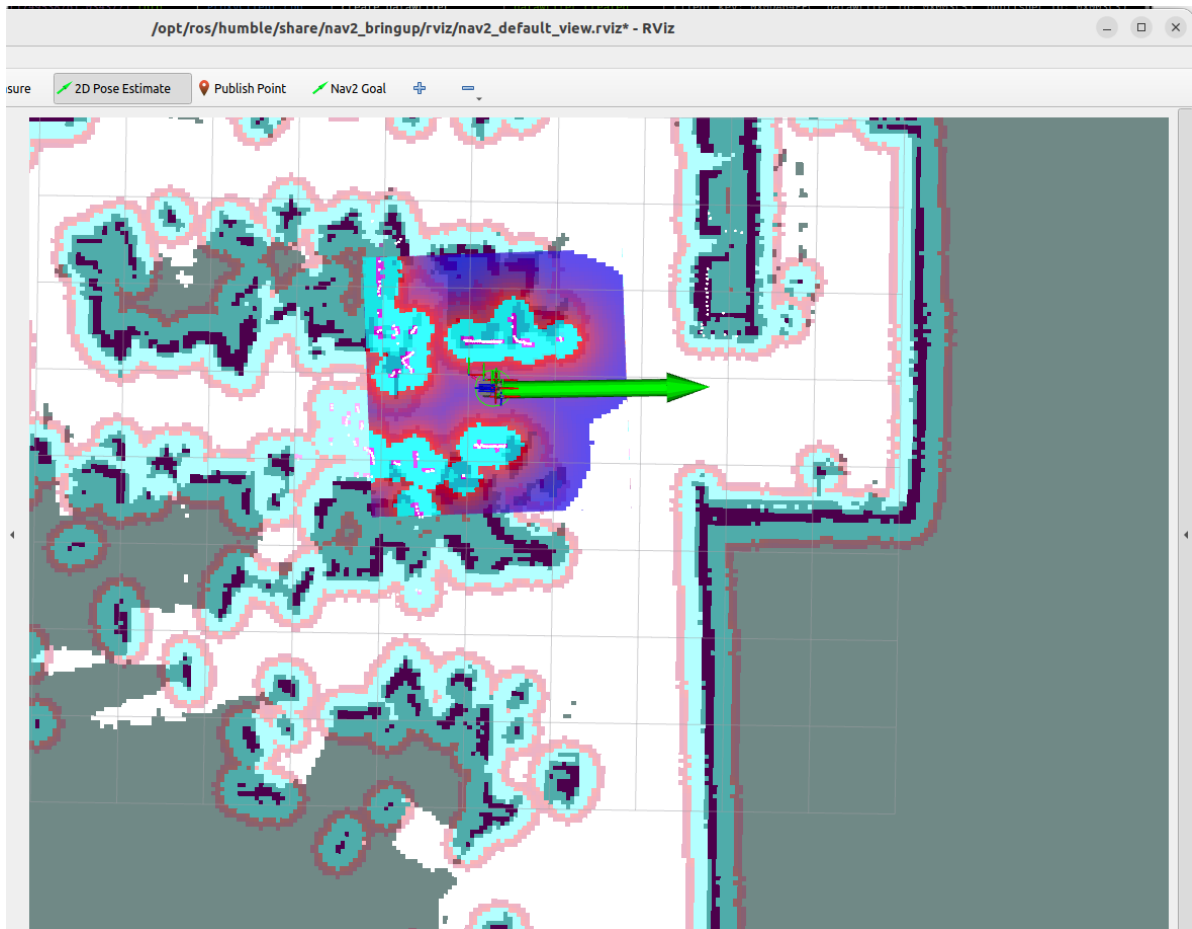
```
ros2 launch M3Pro_navigation base_bringup.launch.py
```

```
ros2 launch M3Pro_navigation navigation2.launch.py
```

- Start rviz on the robot:

```
ros2 launch M3Pro_navigation nav_rviz.launch.py
```

Then, follow the procedure for starting the navigation function to initialize the positioning. This will open the rviz2 visualization interface. Click on **2D Pose Estimate** in the toolbar at the top to enter the selection state. Mark the approximate position and orientation of the robot on the map. After initializing the positioning, the preparation is complete.



4. Running Examples

4.1 Starting the Program

- Run the text interaction node:

```
ros2 run text_chat text_chat
```

- AI agent function on the vehicle's onboard computer:

```
ros2 launch multi_brains llm_agent_control.launch.py text_chat_mode:=True
```

- On the vehicle's onboard computer, open two more terminals and enter the commands to start the navigation function:

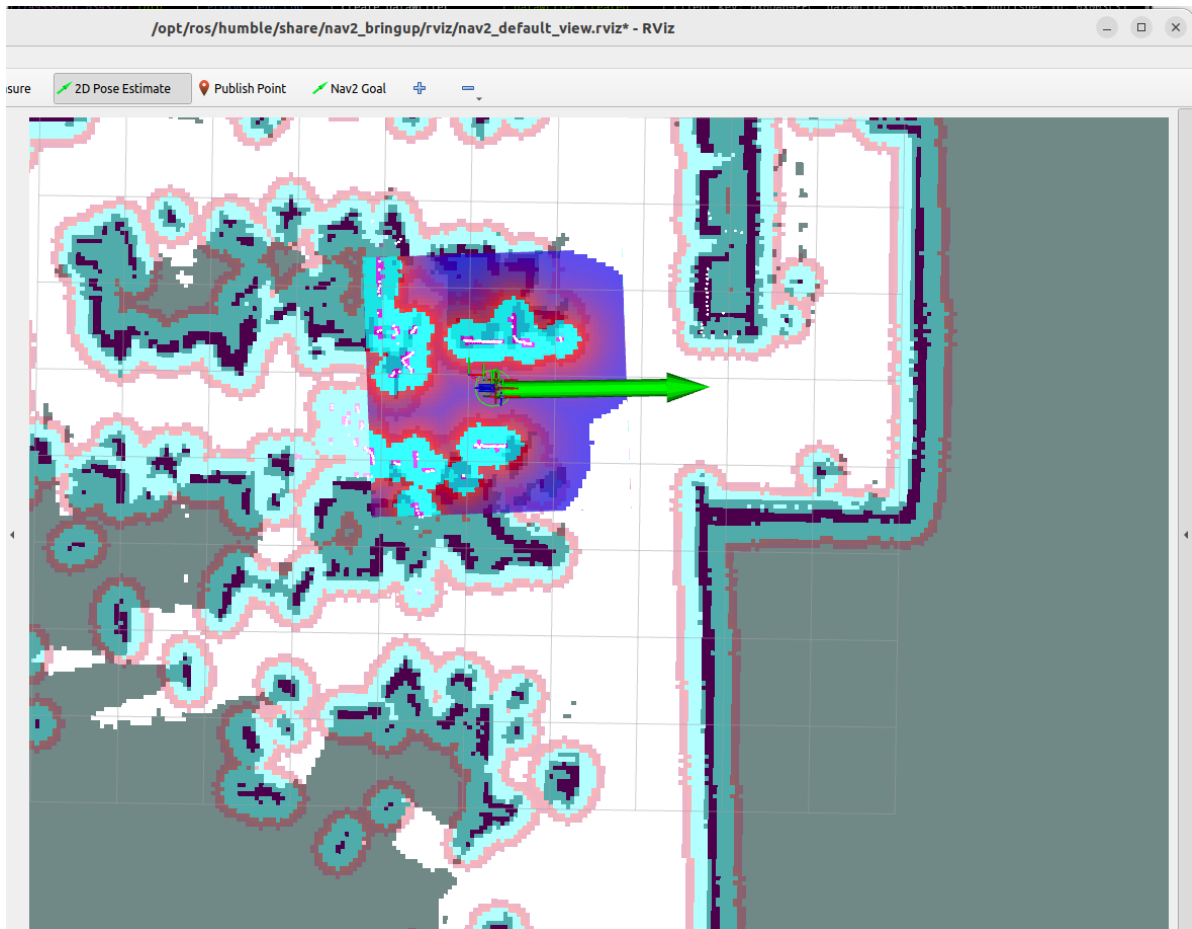
```
ros2 launch M3Pro_navigation base_bringup.launch.py
```

```
ros2 launch M3Pro_navigation navigation2.launch.py
```

- Start rviz on the robot:

```
ros2 launch M3Pro_navigation nav_rviz.launch.py
```

Then, follow the procedure for starting the navigation function to initialize the positioning. This will open the rviz2 visualization interface. Click on **2D Pose Estimate**, enter selection mode, roughly mark the robot's location and orientation on the map. After initialization and localization, the preparation work is complete.

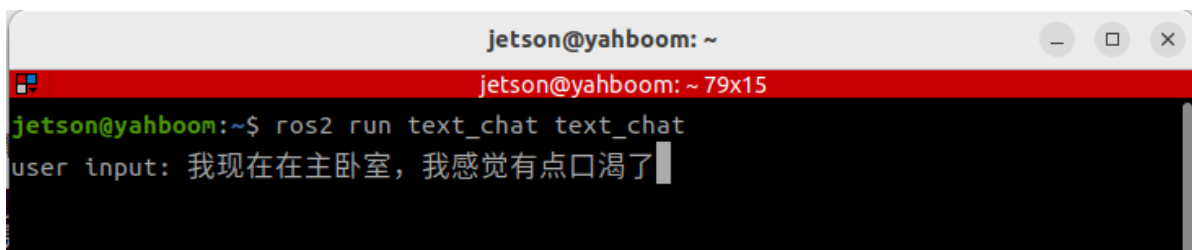


4.2 Test Cases

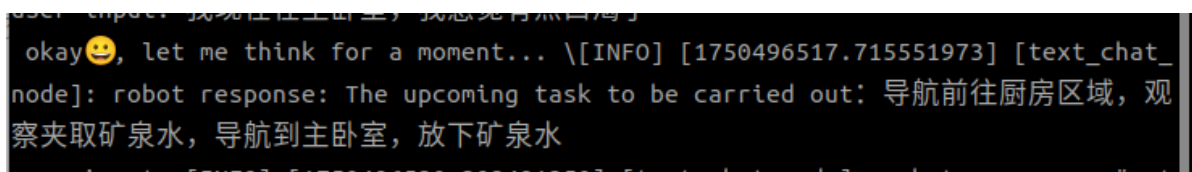
Here are some reference test cases; users can create their own dialogue commands.

- I'm in the master bedroom, and I'm a little thirsty.

Enter the test case in the text interaction terminal:

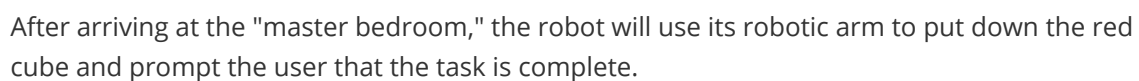


The task steps planned by the decision layer model are as follows:



```
user input: [INFO] [1750496520.302491350] [text_chat_node]: robot response: "action": ['navigation(G)'], "response": 好的，我这就去厨房帮你拿矿泉水。就像一个忠诚的骑士一样，为你的需求而行动！
[INFO] [1750496539.481789151] [text_chat_node]: robot response: "action": ['seewhat()'], "response": 我已经到达厨房了，现在开始寻找矿泉水。让我用我的火眼金睛来定位它！
[INFO] [1750496542.712828693] [text_chat_node]: robot response: "action": ['grasp_obj(350,65,450,310)'], "response": 找到了！矿泉水就在那里，我马上把它夹取过来。就像魔术师一样，变~
```

Note: The robotic arm's factory preset gripping size is for a 3 cm cube. Gripping the bottled water here is only for demonstration purposes. If gripping objects of other sizes, you need to modify the opening angle of the gripper.



```
[INFO] [1750496576.744473877] [text_chat_node]: robot response: "action": ['navigation(A)'], "response": 矿泉水已经稳稳地在我手里了，现在我将它带回主卧室。就像一个快递小哥一样，使命必达！
```

```
[INFO] [1750496595.137145707] [text_chat_node]: robot response: "action": ['putdown()'], "response": 我已经顺利到达主卧室，现在把矿泉水放在这里。请慢用，希望能解你的渴！
```

```
[INFO] [1750496605.264521960] [text_chat_node]: robot response: "action": ['finish_task()'], "response": 任务完成啦！如果你还需要什么帮助，随时告诉我哦。祝你有个美好的一天！
```

